



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Magistrale in Ingegneria Informatica - indirizzo Cybersecurity

Relazione Progetto corso Sistemi distribuiti e cloud computing

Candidato:

Antonio Parise

Mat. 276697

Sommario

1. Introduzione e Obiettivi.....	4
1.1. Contesto e Scopo del Progetto	4
1.2. Requisiti non funzionali.....	5
1.3. Tecnologie e strumenti utilizzati.....	5
2. Architettura Generale del Sistema	7
2.1 Panoramica dell'Architettura Disaccoppiata (Decoupled)	7
2.2. Strategia di Virtualizzazione e Deployment Cloud	8
3. Tecnologie e Strumenti	9
3.1. Livello Dati e Storage:.....	9
3.2. Motore di ricerca e Servizi di Intelligenza Artificiale	10
3.3. Backend e Astrazione dell'Intelligenza Artificiale	10
3.4. Infrastructure as Code	11
4. Gestione delle Identità e Sicurezza.	12
4.1. Architettura del Flusso OAuth2 e OpenID Connect	12
4.2. Separazione delle Responsabilità nel Livello Dati	13
5. Dettagli implementativi: Pipeline di Ingestion e NLP	14
5.1. Ingestion Multiformato e Parsing Testuale	14
5.2. Arricchimento Semantico: Generazione Metadati AI	14
5.3. Strategie di Deduplicazione	16
5.4. Preparazione Dati: Chunking, Embedding e Indicizzazione Vettoriale	18
6. Preparazione Dati: Chunking, Embedding e Indicizzazione Vettoriale.....	20
6.1. Esplorazione dell'Archivio: Ricerca Tradizionale e Paginazione	20
6.2. Retrieval-Augmented Generation (RAG): Architettura del Flusso	20
6.3. Ottimizzazione NLP: Risoluzione delle Allucinazioni da Identificativo	21
6.4. Tracciabilità delle Fonti e Interfacce Utente	22
7. Esperienza Utente e Interfaccia Grafica	23
7.1. Impostazione dell'Interfaccia e Design System	23
7.2. Accesso e Registrazione	23
7.3. Dashboard ed Esplorazione dell'Archivio	24
7.4. Caricamento di un Articolo	25
7.5. Gestione delle Categorie.....	26

7.6	Cronologia Personale e Operazioni CRUD	28
7.7	Scheda dell'Articolo e Chat Contestuale	29
7.8	Ricerca RAG e Tracciabilità delle Fonti	30
7.9	Informazioni di Sistema	31
8.	Ciclo di Vita dell'Applicazione e Virtualizzazione.....	32
8.1.	Containerizzazione e Strategia di Virtualizzazione.....	32
8.2.	Mappatura dei Ruoli Architeturali	32
8.3.	Ambienti di Sviluppo e Gestione del Budget	32
8.4.	Strategia di Deployment per la Release Finale.....	33
9.	Registro delle Interazioni con IA generativa	34
9.1	Infrastruttura Cloud (Azure) e Infrastructure as Code	34
9.2	Sviluppo Backend e Logia Applicativa	37
9.3	Gestione delle Identità e Sicurezza	37
9.3	Sviluppo Frontend e User Experience	38
9.4	Ciclo di Vita, Ottimizzazioni e Deploy	39
10.	Conclusioni e Sviluppi Futuri	41
10.2	Valutazione delle Scelte Architeturali e Cost Management	41
10.3	Sviluppi e Ottimizzazioni Future	42

1. Introduzione e Obiettivi

Il presente documento descrive la progettazione e l'implementazione di un'applicazione cloud-native destinata alla gestione di un archivio di notizie per un quotidiano. In un contesto in cui il volume di informazioni cresce esponenzialmente, si rende necessario un sistema intelligente capace non solo di archiviare, ma anche di analizzare, indicizzare e ricercare contenuti testuali in modo efficiente. A tale scopo, l'applicazione integra un sistema RAG (Retrieval-Augmented Generation) per l'interrogazione avanzata dei dati e la generazione automatica di metadati.

1.1. Contesto e Scopo del Progetto

Le specifiche del progetto impongono la realizzazione di una piattaforma cloud in grado di gestire il ciclo di vita completo degli articoli. Nello specifico, il sistema deve garantire:

- Il supporto di articoli testuali facilmente decodificabili, ad esempio TXT, Markdown o JSON (il supporto a formati più complessi, quali PDF o DOCX, è facoltativo).
- La possibilità di poter inserire manualmente, per ogni articolo caricato nel DB, alcune informazioni descrittive di base, quali titolo, autore, categoria, descrizione e tag.
- Il sistema dovrà generare automaticamente ulteriori metadati tramite tecniche NLP o strumenti di intelligenza artificiale generativa, ad esempio parole chiave, sottotitolo, sintesi del contenuto, lingua dell'articolo o entità rilevanti presenti nel testo.
- Gli articoli caricati dovranno essere suddivisi in frammenti testuali, o chunk, ciascuno dei quali dovrà essere trasformato in una rappresentazione vettoriale tramite un modello di embedding. Gli embedding dei chunk dovranno essere salvati in un database vettoriale o in un sistema equivalente, mantenendo per ogni chunk il riferimento al documento originale, ad esempio tramite un identificativo del documento, un identificativo del chunk e la posizione del frammento nel documento. Si dovrà consentire, durante la fase di retrieval, di risalire dai chunk recuperati agli articoli originali e ai relativi metadati.
- L'utilizzo esclusivo delle soluzioni di calcolo, storage e virtualizzazione fornite dalla piattaforma Microsoft Azure.

Gli obiettivi principali del progetto consistono nella realizzazione di un'architettura disaccoppiata (decoupled), scalabile e resiliente. Si mira a fornire un'interfaccia utente multiplatforma accessibile, supportata da un backend robusto in grado di orchestrare pipeline di intelligenza artificiale avanzate per l'arricchimento semantico dei documenti e la successiva fase di retrieval. Per massimizzare la flessibilità, il sistema impiega una netta separazione delle responsabilità: il client comunica tramite protocollo REST con il backend, il quale funge da orchestratore centrale escludendo l'accesso diretto ai servizi

cloud sottostanti. Infine, l'infrastruttura è stata progettata per garantire elevati standard di sicurezza, delegando la gestione delle identità a soluzioni esterne affidabili.

1.2. Requisiti non funzionali

Oltre ai requisiti esplicitamente richiesti dalla traccia, ho scelto di arricchire il progetto con funzionalità aggiuntive affinché il sistema sia più completo e vicino a un prodotto realmente utilizzabile:

- **Login e Accounting:** è stato implementato un sistema di autenticazione e gestione degli utenti, che consente di associare ogni articolo caricato al proprio autore e di distinguere l'utenza autenticata da quella anonima;
- **Gestione dell'articolo per l'utente che lo carica:** ogni utente autenticato può gestire in autonomia gli articoli da lui pubblicati, con implementazione completa del paradigma CRUD (creazione, lettura, aggiornamento ed eliminazione);
- **Interfaccia grafica dinamica:** il frontend è stato progettato per aggiornarsi reattivamente in base allo stato dell'applicazione e alle azioni dell'utente, senza necessità di ricaricare la pagina;
- **Aggiunta dell'intelligenza artificiale per singolo articolo:** oltre alla ricerca RAG sull'intero archivio, è stata introdotta la possibilità di interrogare l'intelligenza artificiale nel contesto di un singolo articolo, per ottenere approfondimenti o chiarimenti mirati sul suo contenuto.

1.3. Tecnologie e strumenti utilizzati

- **Flutter** è un framework sviluppato da Google per la realizzazione di interfacce cross-platform. Viene impiegato per offrire un'esperienza grafica reattiva e fluida, consentendo l'aggiornamento dinamico dello stato dell'applicazione in tempo reale (come la gestione degli articoli, la compilazione dei metadati e le chat di ricerca RAG) senza richiedere il ricaricamento della pagina. Il framework si basa sul concetto di Widget (che può essere “Stateful” o “Stateless”): l'interfaccia è un albero di widget che viene ridisegnato quando lo stato cambia, e lo stile dei widget può aderire alle linee guida Material di Google o a quelle Cupertino (HIG) di Apple.
- **FastAPI:** è un framework web Python ad alte prestazioni basato sui tipi di dati e sulle operazioni asincrone. Si occupa di gestire la logica applicativa, esporre le API RESTful, orchestrare la pipeline di elaborazione dei documenti (estrazione del testo e suddivisione in chunk), applicare i controlli di sicurezza sugli utenti e comunicare con i modelli di intelligenza artificiale. In confronto a framework usati in ambito enterprise, FastAPI richiede una configurazione minima e consente un deploy rapido.

- **Microsoft Azure:** è una piattaforma cloud che eroga i servizi gestiti per lo storage e l'intelligenza artificiale:
 - Azure Blob Storage: per la conservazione sicura dei file di testo caricati (TXT, Markdown, JSON).
 - Azure AI Search: utilizzato come motore di indicizzazione e database vettoriale per memorizzare e ricercare i chunk e i relativi embedding.
 - Azure OpenAI Service: impiegato per generare gli embedding vettoriali, estratti e metadati automatici (tramite modelli LLM) e formulare le risposte nell'architettura RAG.
- **Azure Bicep:** è un linguaggio dichiarativo di tipo Infrastructure as Code (IaC) sviluppato da Microsoft. Viene utilizzato per definire, automatizzare e distribuire (deploy) in modo ripetibile, sicuro e tracciabile l'intera infrastruttura dei servizi Azure necessari al progetto, eliminando la configurazione manuale tramite portale.
- **Docker:** è un software open source per creare i container, ovvero ambienti di esecuzione isolati che si basano sulla virtualizzazione a livello di sistema operativo. Alla base di un container c'è l'immagine: una collezione di layer indipendenti e sovrapposti, ciascuno dei quali aggiunge funzionalità al precedente. Il dockerfile definisce come costruire l'immagine: da quale immagine partire, quali pacchetti installare, la directory di lavoro, le porte da esporre e il comando di avvio.

2. Architettura Generale del Sistema

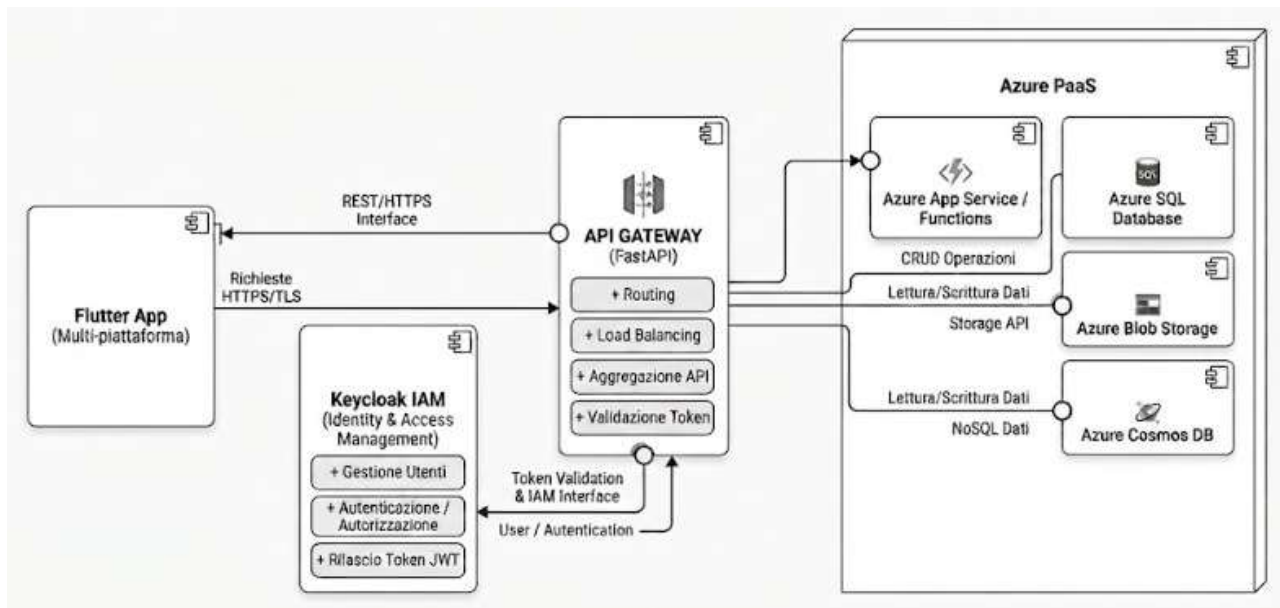


Figura 2.1 Diagramma architetturale

2.1 Panoramica dell'Architettura Disaccoppiata (Decoupled)

La progettazione del sistema è stata guidata dal principio del “Separation of Concerns”, permettendo di avere un’architettura disaccoppiata. Essa, infatti, prevede una netta separazione delle responsabilità attraverso un approccio a macro-layer composto da:

- un Frontend per la presentazione,
- un Backend per l'orchestrazione
- un ecosistema di Servizi Cloud per la persistenza e l'elaborazione avanzata.

Questo approccio consente di far evolvere, scalare e mantenere i singoli componenti in modo indipendente, riducendo il rischio che colli di bottiglia o malfunzionamenti in uno strato possano compromettere l'intero ciclo di vita dell'applicazione.

L'architettura così composta ha permesso di massimizzare la flessibilità del sistema in un contesto cloud-native. Il frontend, operando come client autonomo e agnostico rispetto all'infrastruttura sottostante, comunica esclusivamente tramite protocollo HTTP/REST (scambiando payload in formato JSON) con il backend. Quest'ultimo, sviluppato in Python tramite il framework FastAPI, non si limita a esporre dati, ma funge da orchestratore centrale (API Gateway) della logica di business e delle complesse pipeline di intelligenza artificiale destinate all'estrazione e alla manipolazione semantica delle notizie. Da un punto di vista della sicurezza e della topologia di rete, i servizi Azure sottostanti operano come layer di persistenza e computazione specializzata, accessibili esclusivamente tramite le API del backend. Questo design pattern vieta categoricamente

al client di interrogare direttamente il database o i servizi di storage, garantendo un controllo centralizzato, robusto e imperativo sulla sicurezza e sui flussi di dati.

2.2. Strategia di Virtualizzazione e Deployment Cloud

In accordo alla richiesta progettuale che impone di utilizzare i servizi messi a disposizione da Microsoft Azure, il sistema affida il proprio strato computazionale alle tecnologie di virtualizzazione offerte dalla piattaforma.

Le componenti non erogate in modalità Platform-as-a-Service (PaaS), in particolare il Backend (FastAPI) e il gestore delle identità (Keycloak), sono state interamente containerizzate avvalendosi della tecnologia Docker. Questa astrazione a livello di sistema operativo permette di incapsulare l'ambiente di esecuzione e le relative dipendenze, rendendo l'applicativo idoneo al deployment su istanze di calcolo cloud (come Azure Virtual Machines o Azure Container Instances), garantendo riproducibilità degli ambienti e agevolando lo scaling.

3. Tecnologie e Strumenti

Questo capitolo concerne l'analisi delle scelte tecnologiche per ciascun livello che compone l'architettura:

3.1. Livello Dati e Storage:

A questo livello architetturale il cloud è stato utilizzato per favorire la separazione tra lo storage di file fisici e l'archiviazione dei dati favorendo la persistenza dei dati e scartando totalmente l'uso di database relazionali (SQL).

- **Azure Blob Storage:** adottato per l'archiviazione dei file originali e della copertina caricati dagli utenti. Questa scelta progettuale ha diverse motivazioni, in particolare per la scalabilità virtualmente illimitata e per l'elevata affidabilità per il salvataggio dei dati non strutturati; inoltre, è da notare che i block blob sono ottimizzati per lo streaming e l'archiviazione di documenti file multimediali.
- **Azure Cosmos DB:** la gestione dei metadati, manuali e generati dai modelli di IA, è totalmente affidata a Cosmos DB. L'uso di un database documentale nativo JSON risulta superiore rispetto a un approccio relazionale rigido per via dello schema disomogeneo degli articoli. La scelta di impiegare la modalità Serverless è dovuta alla necessità di ottimizzare i costi operativi, cosicché i consumi avvengano solo durante le effettive operazioni, nel caso in cui l'utente possegga dei limiti di budget.

```
{
  "id": "ff90d859-140b-43be-910f-28fb161340f2",
  "user_id": "69f23574-1691-4ed3-b0bc-21366d2e0d14",
  "cover_url": "https://pcloud3vwgrko3on.blob.core.windows.net/articles-image/ff90d859-140b-43be-910f-28fb161340f2",
  "blob_url": "https://pcloud3vwgrko3on.blob.core.windows.net/articles-raw/ff90d859-140b-43be-910f-28fb161340f2",
  "uploaded_at": "2026-09-03T13:44:49.028184Z",
  "manual": {
    "title": "Le 12 best practice dell'ingegneria dei sistemi di intelligenza artificiale",
    "author": "Vincenzo Calabrò",
    "category": [
      "CyberSecurity",
      "Intelligenza Artificiale"
    ],
    "description": "Il documento propone un'analisi dell'aggiornamento sulle 12 pratiche di base per l'ingegneria dei sistemi di intelligenza artificiale",
    "tags": [
      "Intelligenza Artificiale",
      "Best Practice",
      "Sicurezza",
      "Data Poisoning"
    ]
  }
},
```

Figura 3.1 screenshot file JSON Cosmos DB

3.2. Motore di ricerca e Servizi di Intelligenza Artificiale

Una delle richieste fondamentali era il sistema RAG, la scelta per l'implementazione è ricaduta su due servizi specializzati dell'ecosistema Azure.

- **Azure AI Search:** utilizzato per l'indicizzazione dei frammenti di testo (chunk) e la successiva ricerca semantica, poiché gestisce in un unico servizio funzionalità di ricerca vettoriale e di ricerca full-text eliminando la complessità operativa di dover sfruttare i database vettoriali di terze parti. Per prevenire i conflitti nei criteri di indicizzazione la struttura dell'indice è stata ingegnerizzata definendo esplicitamente i campi testuali per le query full-text e un campo vettoriale a 1536 dimensioni.
- **Azure OpenAI Service:** l'integrazione dei modelli linguistici per la generazione dei metadati e per il motore RAG si basa su Azure OpenAI Service, poiché rappresenta un valido compromesso tra capacità elaborativa ed efficienza. A causa delle restrizioni dovute ad Azure for Students, il deployment di questi modelli è stato l'unico step non automatizzato attraverso Bicep nel momento della definizione dei servizi Azure.

3.3. Backend e Astrazione dell'Intelligenza Artificiale

Il livello responsabile della gestione della sicurezza e dello smistamento delle richieste è stato sviluppato in Python.

- **FastAPI e Pydantic:** il suo supporto nativo alla programmazione asincrona è stato la caratteristica cruciale per la scelta, poiché permette che i thread del server non vengano bloccati durante le lunghe attese per l'upload su Blob Storage o per le risposte dell'AI. Inoltre, Pydantic garantisce una validazione formale e rigorosa dei dati in ingresso e in uscita.
- **LangChain:** strumento rivelatosi cruciale non solo per la costruzione delle pipeline di intelligenza artificiale, spesso di elevata complessità, ma anche per l'estrazione dei metadati. Infatti, tramite l'approccio LCEL e la funzione "with_structured_output", il backend è in grado di forzare i LLM a restituire risultati deterministici in un formato JSON, in modo tale che aderisca agli schermi Pydantic (BaseModel) del progetto, azzerando gli errori di parsing.

```
from pydantic import BaseModel

class RagSearchQuery(BaseModel):
    question : str
```

Figura 3.2 Base model

```

structured_llm = llm.with_structured_output(MetadataIA)
prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "Sei un assistente editoriale esperto in analisi testuale e NLP. "
        "Il tuo compito è analizzare articoli giornalistici ed estrarre informazioni chiave in modo oggettivo e fedele al testo originale.\n\n"
        "REGOLE DI ESTRAZIONE:\n"
        "- RIASSUNTO: genera un riassunto di almeno 3-4 frasi corpose, basato ESCLUSIVAMENTE sui fatti presenti nel testo. "
        "Non aggiungere interpretazioni, opinioni o informazioni non presenti nell'articolo.\n"
        "- SOTTOTITOLO: genera un sottotitolo descrittivo e accattivante, ma NON sensazionalistico o fuorviante rispetto al contenuto reale (no clickbait).\n"
        "- KEYWORDS: estrai parole chiave rilevanti e distinte tra loro, evitando sinonimi ridondanti.\n"
        "- ENTITÀ: estrai solo entità esplicitamente nominate nel testo (persone, luoghi, organizzazioni), specificandone il tipo. Non inferire entità non citate.\n"
        "#- CATEGORIE: suggerisci solo categorie coerenti con il contenuto effettivo, evitando categorie generiche se non pertinenti.\n"
        "- LINGUA: rileva la lingua originale del testo, non tradurre.\n"
        "- Se il testo fornito è troppo breve o ambiguo per un'estrazione affidabile, genera comunque i campi ma mantieni "
        "riassunto e sottotitolo aderenti a ciò che è effettivamente presente, senza inventare dettagli mancanti."
    ),
    (
        "human",
        "Analizza il seguente testo ed estrai i metadati richiesti:\n\n{text_content}"
    )
])

```

Figura 3.3 Prompt

3.4. Infrastructure as Code

La configurazione manuale tramite portale web è causa di errori umani non volontari, per evitarlo si è deciso di automatizzare interamente il provisioning delle risorse cloud tramite Azure Bicep. Quest'ultima, soluzione nativa per l'ecosistema Microsoft, garantisce la riproducibilità totale degli ambienti cloud e soprattutto non richiede la gestione complessa di file di stato locali o remoti, allineando pienamente il progetto agli standard avanzati richiesti e alle best practice di programmazione in merito.

```

targetScope = 'subscription'

// PARAMETRI

@description('Nome del Resource Group da creare')
param resourceGroupName string = 'progettocloud-rg'

@description('Regione Azure per tutte le risorse')
param location string = 'italynorth'

@description('Regione Azure per Azure OpenAI. Impostata a italynorth per conformità con la policy Azure fo')
param openAiLocation string = 'italynorth'

@description('Prefisso breve per i nomi delle risorse (3-8 caratteri, solo minuscole e numeri)')
@minLength(3)
@maxLength(8)
param prefix string = 'pcloud'

// RESOURCE GROUP

resource rg 'Microsoft.Resources/resourceGroups@2023-07-01' = {
  name: resourceGroupName
  location: location
  tags: {
    project: 'Archivio Notizie'
    course: 'Sistemi Distribuiti e Cloud Computing'
    managedBy: 'Bicep IaC'
  }
}

```

Figura 3.4 esempio codice bicep

4. Gestione delle Identità e Sicurezza.

La protezione degli endpoint sensibili e la gestione della cronologia personale degli utenti richiedono un solido strato di autenticazione e autorizzazione. La scelta architetturale di questo livello è stata presa dopo un'attenta analisi delle alternative: visto il vincolo sull'utilizzo dell'ecosistema Microsoft, è stata valutata l'adozione di un approccio custom basato sulla generazione interna di JSON Web Tokens (JWT) da abbinare a Cosmos DB; successivamente è stato preso in considerazione l'utilizzo di Microsoft Authentication Library (MSAL) integrato con Azure Entra ID B2C.

Il primo approccio, però, viola il principio fondamentale della sicurezza informatica: “don't roll your own crypto”, secondo il quale è sconsigliato gestire internamente le password e le policy crittografiche. Il secondo invece, presenta delle rigide restrizioni sui permessi di amministrazione imposti dalla sottoscrizione universitaria Azure for Students, facendo sì che il provisioning di un nuovo tenant B2C presentasse forti rischi di blocco operativo e quindi impossibilità di creare nuove directory.

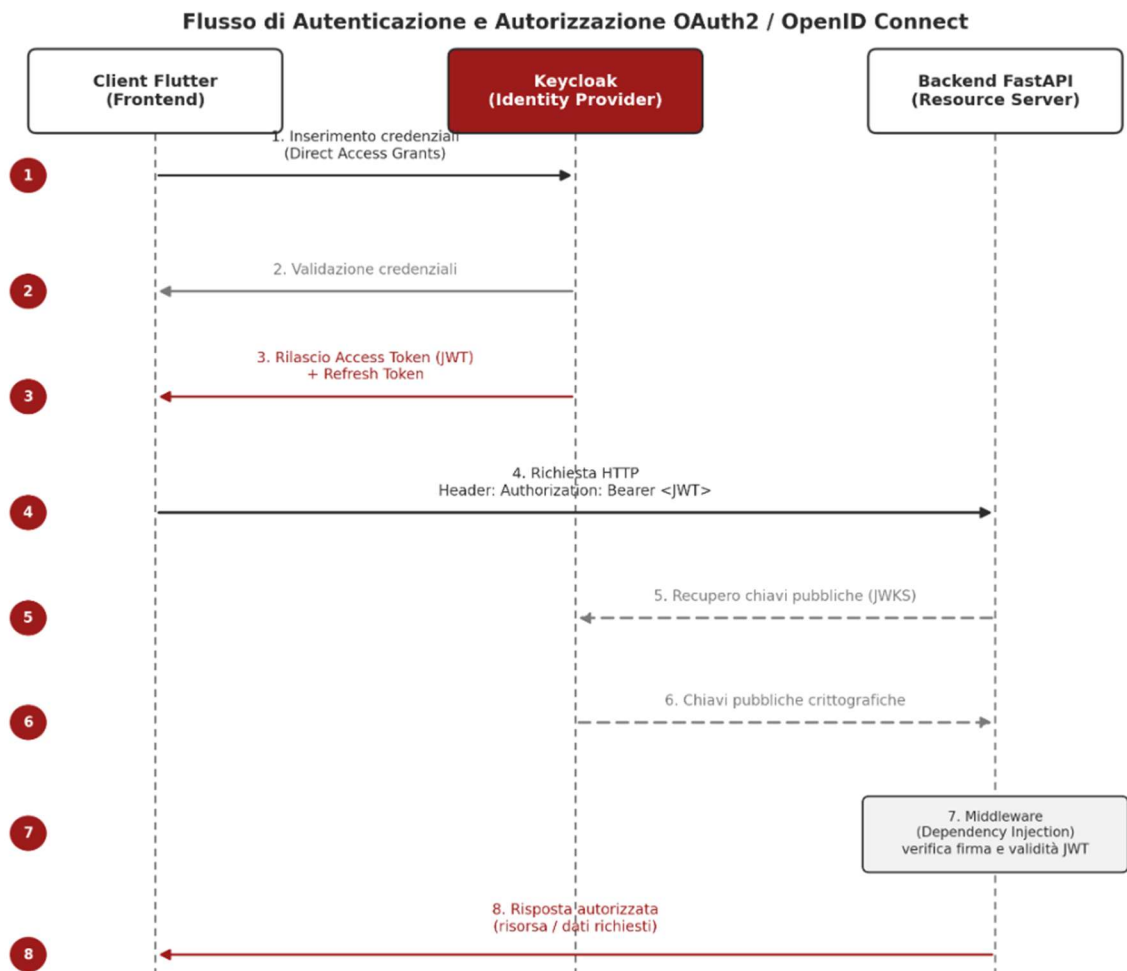
In definitiva, per rispettare le best practice di sicurezza delegando l'autenticazione a un Identity Provider certificato, e per aggirare le limitazioni della sottoscrizione Azure e quindi far continuare la user Experience senza uscire dal flusso applicativo, si è optato per integrazione di Keycloak. Quest'ultimo è distribuito tramite container Docker, motivo per il quale la sua esecuzione è stata pensata per essere ospitata sulle risorse di virtualizzazione di calcolo di Azure o in ambiente locale per le fasi di sviluppo e demo.

4.1. Architettura del Flusso OAuth2 e OpenID Connect

L'interazione tra Keycloak, il frontend Flutter e il backend FastAPI si basa sui protocolli standard OAuth2.0 e OpenID Connect. In ottemperanza al pattern architetturale disaccoppiato, il backend fastAPI agisce come validatore e non gestisce mai le password in chiaro. Il flusso implementato si articola come segue:

- Autenticazione: l'utente inserisce le credenziali nel client Flutter. Per mantenere l'utente all'interno dell'interfaccia personalizzata dell'applicazione senza reindirizzamenti esterni, su Keycloak è stato abilitato il flusso Direct Access Grants.
- Rilascio Token: se le credenziali sono valide, Keycloak restituisce al frontend un Access Token (JWT) e un Refresh Token. Per accomodare le esigenze dell'applicazione, la validità dell'Access Token e della sessione SSO in Keycloak è stata estesa rispetto ai parametri di default, per evitare disconnessioni troppo frequenti.
- Autorizzazione e Validazione: il frontend inietta l'Access Token nell'header “Authorization: Bearer” di ogni successiva chiamata http verso FastAPI. Il backend, avvalendosi della libreria “python-jose”, verifica matematicamente la validità e l'integrità del token in ingresso tramite un middleware di “Dependency

Injection”. Le chiavi pubbliche crittografiche del realm (JWKS) non vengono ricaricate a ogni richiesta, ma mantenute in una cache locale con scadenza temporale, protetta da un lock per evitare download concorrenti; la cache viene invalidata automaticamente quando arriva un token firmato con una chiave sconosciuta, così da gestire in modo trasparente la rotazione delle chiavi del realm.



4.2. Separazione delle Responsabilità nel Livello Dati

L'inclusione di Keycloak non ha intaccato il principio di *Separation of Concerns*; infatti, i database e lo storage (Azure Cosmos DB e Blob Storage) rimangono agnostici rispetto all'infrastruttura di sicurezza. L'estrazione dell'identificativo univoco dell'utente (utile per tracciare la proprietà degli articoli o limitare le operazioni CRUD come la cancellazione e la modifica) avviene esclusivamente nel livello di routing (FastAPI). Tale stringa identificativa viene passata pulita ai servizi inferiori, preservando l'isolamento dei layer software.

5. Dettagli implementativi: Pipeline di Ingestion e NLP

La fase di ingestion rappresenta il punto di ingresso dell'informazione nel sistema e il ponte tra l'archiviazione tradizionale e il mondo dell'Intelligenza Artificiale. Questa complessa pipeline, orchestrata dal backend, esegue in sequenza operazioni di parsing testuale, validazione dell'originalità del documento, persistenza su storage, arricchimento semantico e preparazione dei dati vettoriali (chunking ed embedding). Per garantire prestazioni elevate sotto carico, l'intera pipeline è stata implementata sfruttando i paradigmi della programmazione asincrona (tramite `async` e `await`), prevenendo il blocco del server durante le operazioni I/O-bound verso i servizi cloud.

5.1. Ingestion Multifornito e Parsing Testuale

L'endpoint di caricamento (esposto tramite metodo HTTP POST su `/articles/upload`) è il “direttore d'orchestra” del flusso: riceve il file, l'immagine di copertina e i metadati manuali, e coordina ogni fase successiva.

Il primo passo, dopo il controllo di esistenza del titolo, è l'estrazione del contenuto testuale grezzo (parsing), necessaria per alimentare sia i controlli anti-duplicazione sia i modelli AI. Oltre ai formati testuali standard (TXT, Markdown, JSON), elaborati tramite le librerie built-in di Python, il sistema soddisfa il requisito facoltativo di progetto integrando il supporto per formati complessi: i documenti Microsoft Word (.docx) vengono elaborati tramite la libreria `python-docx`, estraendone il testo paragrafo per paragrafo, mentre i documenti PDF vengono analizzati pagina per pagina avvalendosi di `PyPDF2`. Solo dopo aver superato entrambi i controlli di deduplicazione, il file grezzo viene effettivamente caricato su Azure Blob Storage garantendo la persistenza dei dati. Tale scelta progettuale è stata effettuata per evitare di archiviare permanentemente contenuti che risulteranno duplicati.

Il testo estratto diventa poi l'input sia per la generazione dei metadati AI, sia per la fase di chunking: il documento viene suddiviso in frammenti tramite `RecursiveCharacterTextSplitter` di `LangChain`, con una dimensione di 500 caratteri per chunk e un overlap di 50 caratteri tra chunk consecutivi, per preservare il contesto semantico ai bordi di ogni segmento.

5.2. Arricchimento Semantico: Generazione Metadati AI

Una delle specifiche tecniche imponeva l'estrazione automatica di metadati descrittivi dall'articolo. Per realizzare questa feature, si è scelto di non affidarsi al parsing manuale di testo libero generato dall'IA, ma di implementare un approccio rigoroso basato sulla funzionalità di Function Calling (o Structured Output). L'architettura sfrutta il framework `LangChain` in combinazione con i modelli `gpt-4o-mini` e `text-embedding-ada-002` erogati da Azure OpenAI (configurati tramite deployment dedicato). È stato definito uno schema dati esplicito tramite la libreria `Pydantic` (modello `MetadataAI`), comprendente campi

opzionali quali sottotitolo, riassunto, parole chiave, categorie suggerite, lingua ed entità nominate. Passando questo modello formale al metodo `with_structured_output` di `LangChain`, si costringe in modo deterministico LLM, configurato con `temperature=0.2` per ridurre le “allucinazioni” e favorire un'analisi oggettiva, a restituire esclusivamente un oggetto JSON valido e tipizzato.

```
class ManualMetadata(BaseModel): 2 usages 2 Parigi07
    """Metadati inseriti manualmente dall'utente al momento dell'upload."""
    title: Optional[str] = Field(None, description="Titolo dell'articolo")
    author: Optional[str] = Field(None, description="Autore o fonte dell'articolo")
    category: Optional[List[str]] = Field(None, description="Categoria tematica (es. Politica, Sport)")
    description: Optional[str] = Field(None, description="Breve descrizione manuale")
    tags: Optional[List[str]] = Field(default_factory=list, description="Lista di tag liberi")

class MetadataIA(BaseModel): 5 usages 2 Parigi07
    """Metadati generati da un modello IA o tramite tecniche NLP"""
    subtitle: Optional[str] = Field(None, description="Sottotitolo dell'articolo")
    keywords: Optional[List[str]] = Field(None, description="Lista di parole chiave")
    category: Optional[List[str]] = Field(None, description="Lista di categorie")
    language: Optional[str] = Field(None, description="Lingua")
    summary: Optional[str] = Field(None, description="Breve sintesi dell'articolo")
    entities: Optional[List[str]] = Field(None, description="Entità rilevanti come persone, luoghi o organizzazioni")
```

Figura 5.1 modello Pydantic

Il prompt di sistema, inoltre, impone regole di estrazione rigorose: il riassunto deve basarsi esclusivamente sui fatti presenti nel testo, le entità devono essere esplicitamente nominate nel documento (senza inferenze), e la lingua rilevata non deve essere tradotta. Questo approccio prompt-engineered riduce ulteriormente il rischio di output fuorvianti o inventati.

```
prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "Sei un assistente editoriale intelligente. L'utente sta attualmente leggendo l'articolo con ID: '{current_article_id}'.\n\n"
        "Ho cercato nell'intero archivio e ho trovato questi frammenti correlati:\n\n"
        "CONTESTO GLOBALE:\n{context}\n\n"
        "REGOLE DI RISPOSTA:\n"
        "1. Rispondi basandoti ESCLUSIVAMENTE sulle informazioni presenti nel CONTESTO GLOBALE. Non inventare fatti, nomi, numeri o eventi non presenti nei frammenti.\n"
        "2. Se l'utente chiede spiegazioni sull'articolo in lettura (ID '{current_article_id}'), usa prioritariamente i frammenti con quello stesso Documento ID.\n"
        "3. Se l'utente chiede di articoli simili o cosa altro c'è in archivio sul tema, guarda i frammenti con Documento ID DIVERSO da '{current_article_id}'.\n"
        "Se e SOLO SE ne trovi di realmente pertinenti alla domanda, rispondi con entusiasmo indicando il Titolo del documento e un breve riassunto fedele al contenuto.\n"
        "4. Se nel CONTESTO GLOBALE non ci sono informazioni pertinenti alla domanda (né nell'articolo corrente né altrove), dillo chiaramente: "\n"
        "Mi dispiace, non ho trovato informazioni pertinenti nell'archivio." Non forzare una risposta positiva se il contesto non la supporta davvero.\n"
        "5. Cita sempre il Documento Title quando riporti un'informazione specifica.\n"
        "6. Rispondi nella stessa lingua della domanda dell'utente."
    ),
    (
        "human",
        "Domanda dell'utente: {question}"
    )
])
```

Figura 5.2 prompt

Per contenere i costi in fase di sviluppo e test, il sistema espone anche una modalità mock (`TEST_MODE`), che restituisce metadati fittizi senza consumare crediti API; scelta utile sia in fase di debug sia per dimostrazioni offline del sistema.

5.3. Strategie di Deduplicazione

Per garantire l'originalità dell'archivio ed evitare l'inquinamento del database vettoriale, è stato ingegnerizzato un robusto sistema di deduplicazione basato su un pattern a “due cancelli”, eseguito interamente prima che il file venga caricato su Blob Storage o che vengano innescate le operazioni di arricchimento NLP sull'intero testo. Questo meccanismo blocca l'upload di articoli già presenti ottimizzando contemporaneamente il consumo di token (pay-per-use).

- **Cancello 1 (Deduplicazione Esatta):** un controllo deterministico, veloce e a costo zero. Prima di qualsiasi elaborazione, il backend interroga Azure Cosmos DB verificando l'esistenza di un articolo con il medesimo titolo. In caso di esito positivo, la richiesta viene respinta con un errore HTTP 400.

```
def check_title_exists(title: str) -> bool:
    """Funzione di check (primo blocco) sulla similarità dei titoli
    evitando che due articoli abbiano stesso titolo
    :param title: titolo file inserito dall'utente
    :return boolean:
    """

    query = "SELECT * FROM c WHERE c.manual.title = @title"
    parameters = [
        {"name": "@title", "value": title}
    ]

    try:
        query_items = articles_container.query_items(query=query, parameters=parameters, enable_cross_partition_query=True)
        results = list(query_items)
        print(f"Trovati duplicati: {len(results)}")

        if len(results) > 0:
            return True
        return False
    except CosmosHttpResponseError as e:
        print(f"Errore durante la query su Cosmos DB: {e}")
        raise
```

Figura 5.3 check_title function

- **Cancello 2 (Similarità Semantica):** un controllo per individuare documenti concettualmente identici ma con titoli diversi. Invece di analizzare l'intero documento (con spreco di token), il sistema estrae un frammento “spia”, lo vettorizza tramite Azure OpenAI Embeddings e lo invia ad Azure AI Search per un confronto di similarità vettoriale. Se il motore di ricerca rileva una similarità semantica superiore a una certa soglia, il documento viene classificato come duplicato semantico e la richiesta viene respinta, prima ancora che il file venga caricato su storage o che vengano avviate le costose operazioni di generazione metadati AI e chunking.


```

async def check_similarity(vector: list[float | int ], threshold: float | int = 0.90) -> bool : 1 usage &Parise07 "
'''
~~~~~
Funzione che restituisce uno score di similarità di un articolo con gli altri presenti nell'archivio e restituisce
false nel momento in cui un file ha un threshold superiore al 90 percento di default
:param vector:
:param threshold:
:return:
'''
~~~~~
if settings.TEST_MODE:
    print("❌ MOCK MODE: Controllo similarità vettoriale saltato.")
    return False
vector_query = VectorizedQuery(
    vector= vector,
    k_nearest_neighbors=1,
    fields="embedding"
)
local_search_client = SearchClient(
    endpoint=settings.AZURE_SEARCH_ENDPOINT,
    index_name=settings.AZURE_SEARCH_INDEX_NAME,
    credential=AzureKeyCredential(settings.AZURE_SEARCH_ADMIN_KEY)
)
try:

```

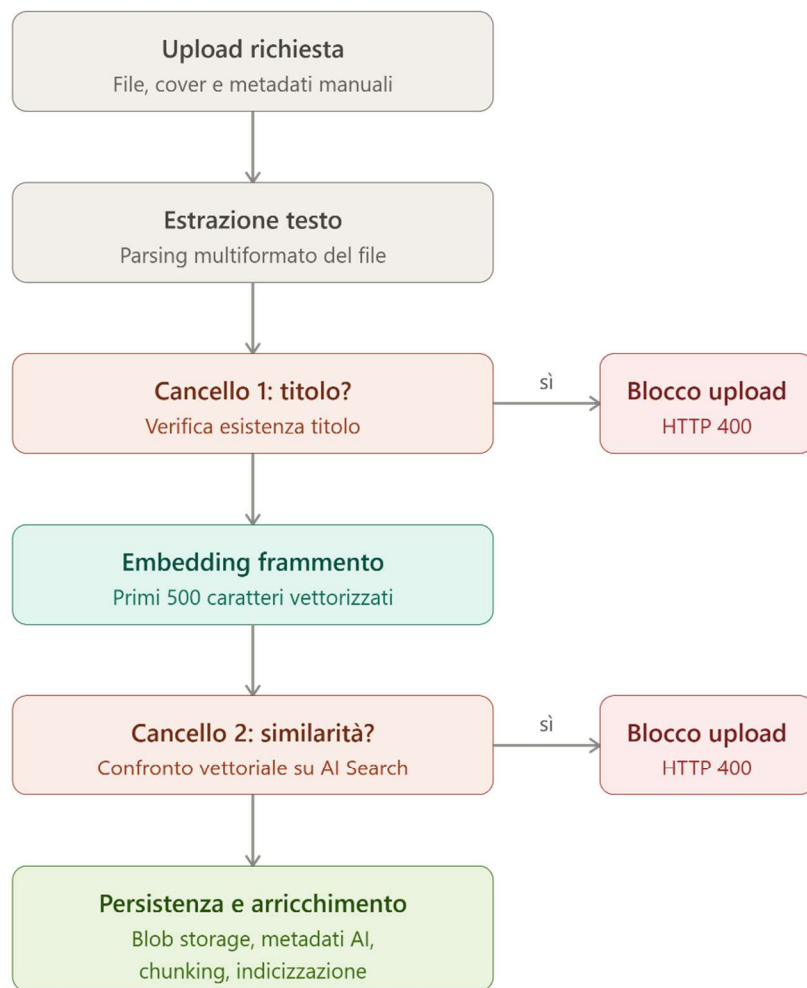
```

    local_search_client = SearchClient(
        endpoint=settings.AZURE_SEARCH_ENDPOINT,
        index_name=settings.AZURE_SEARCH_INDEX_NAME,
        credential=AzureKeyCredential(settings.AZURE_SEARCH_ADMIN_KEY)
    )
    try:
        async with local_search_client:
            # Eseguiamo la ricerca
            results = await local_search_client.search(
                search_text=None,
                vector_queries=[vector_query],
                top=1
            )
            async for result in results:
                score = result.get("@search.score", 0)
                print(f" Punteggio di similarità rilevato: {score}")

                if score >= threshold:
                    return True
            return False
    except Exception as e:
        print(f"Errore durante il controllo di similarità: {e}")
        return False

```

Solo dopo aver superato entrambi i cancelli, la pipeline procede con la persistenza del file (Blob Storage), la generazione dei metadati AI, il chunking del testo e l'indicizzazione vettoriale dei singoli frammenti su Azure AI Search.



5.4. Preparazione Dati: Chunking, Embedding e Indicizzazione Vettoriale

L'ultima fase della pipeline prepara il documento per il futuro motore RAG. Anche in questo caso, le operazioni sono orchestrate in modo asincrono nel backend.

- **Chunking:** il testo validato non viene inviato interamente al database vettoriale. Viene invece segmentato utilizzando il componente `RecursiveCharacterTextSplitter` di `LangChain`. I parametri scelti (`chunk_size=500` e `chunk_overlap=50`) garantiscono frammenti agili da elaborare per l'LLM, mantenendo al contempo una sovrapposizione semantica sui margini per non interrompere concetti logici a metà frase.
- **Vettorizzazione (Embedding):** ciascun chunk viene convertito asincronamente in una rappresentazione matematica tramite il modello specializzato `text-embedding-ada-002`. Il risultato è una lista di vettori densi a 1536 dimensioni.

- **Indicizzazione:** attraverso l'uso del client asincrono SearchClient, testi e vettori vengono accoppiati (assegnando a ciascuno un chunk_id univoco collegato al documento originale) e caricati in blocco (batch) sull'indice di Azure AI Search, rendendo il documento ufficialmente pronto per le interrogazioni semantiche in linguaggio naturale.

6. Preparazione Dati: Chunking, Embedding e Indicizzazione Vettoriale

Il soddisfacimento del requisito relativo all'interrogazione dell'archivio ha richiesto l'ingegnerizzazione di due flussi di ricerca paralleli: una ricerca testuale “tradizionale” per l'esplorazione generica e un motore RAG (Retrieval-Augmented Generation) avanzato per l'interrogazione in linguaggio naturale. Entrambi i flussi sono esposti dal backend FastAPI e consumati dal client Flutter, garantendo un'esperienza utente fluida e reattiva.

6.1. Esplorazione dell'Archivio: Ricerca Tradizionale e Paginazione

Per garantire un accesso rapido e strutturato ai dati, bypassando i costi computazionali dell'intelligenza artificiale quando non strettamente necessaria, è stata implementata un'API di ricerca tradizionale (endpoint `/search/generic`). A livello di database, questa funzionalità interroga Azure Cosmos DB utilizzando l'operatore NoSQL `CONTAINS` con flag *case-insensitive*, permettendo di rintracciare occorrenze parziali di stringhe all'interno di campi multipli (titolo, descrizione, autore, tag e categoria).

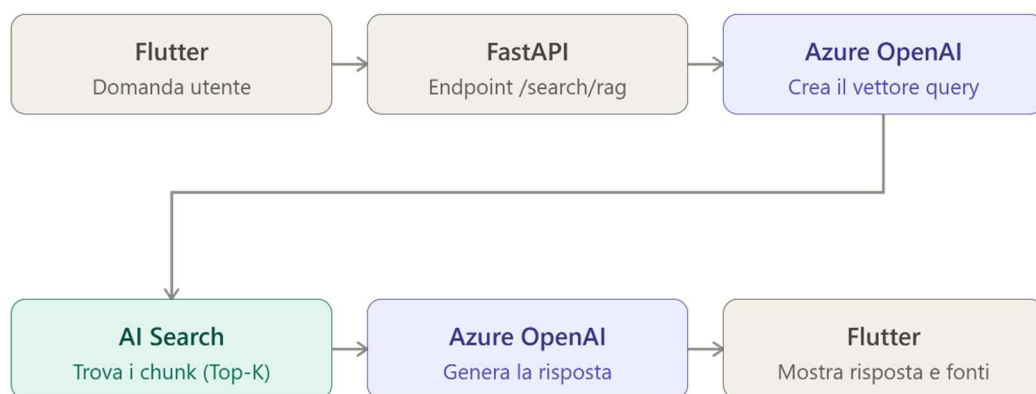
Parallelamente, per supportare la visualizzazione a lista nel frontend (dashboard principale), è stato sviluppato l'endpoint `GET /articles`. Per garantire scalabilità e prestazioni ottimali in presenza di volumi di dati elevati, questo endpoint implementa la **paginazione nativa** per supportare un'interfaccia a scorrimento infinito (*Infinite Scroll*). Tramite l'uso combinato dei parametri `OFFSET` e `LIMIT` (mappati sui query parameters `skip` e `limit` di FastAPI), accoppiati a un ordinamento cronologico sul campo `uploaded_at` (`ORDER BY c.uploaded_at DESC` nella modalità predefinita), il backend fornisce al client porzioni controllate di dati senza sovraccaricare la memoria.

```
if decreasing:
    query += " ORDER BY c.uploaded_at ASC OFFSET @skip LIMIT @limit"
else:
    query += " ORDER BY c.uploaded_at DESC OFFSET @skip LIMIT @limit"
```

6.2. Retrieval-Augmented Generation (RAG): Architettura del Flusso

Il motore RAG costituisce il cuore innovativo del progetto, rispondendo al requisito che impone la capacità di porre domande complesse. Il processo è diviso in due macro-fasi logiche: *Retrieval* (Recupero) e *Generation* (Generazione).

- **Vettorizzazione della Query:** la domanda dell'utente in linguaggio naturale viene inviata all'endpoint `/search/rag`, protetto dal sistema di autenticazione per evitare abusi sui consumi API. Il backend converte asincronamente la domanda testuale in un vettore numerico a 1536 dimensioni utilizzando il medesimo modello `text-embedding-ada-002` impiegato in fase di ingestion.
- **Ricerca Vettoriale (Retrieval):** il vettore risultante viene passato ad Azure AI Search tramite la classe `VectorizedQuery` dell'SDK Python. Il motore di ricerca calcola la distanza matematica (similarità semantica) tra la domanda e tutti i frammenti indicizzati, restituendo i *Top-K* chunk più pertinenti e i relativi punteggi di accuratezza (score).
- **Orchestrazione del Prompt e Generazione:** i testi recuperati vengono aggregati e inseriti dinamicamente in un template (`ChatPromptTemplate` di LangChain) che funge da “contesto limitato” per l'LLM (`gpt-4o-mini`). L'istruzione di sistema (*System Prompt*) vincola rigidamente l'Intelligenza Artificiale a rispondere basandosi *esclusivamente* sulle informazioni fornite, azzerando l'uso di conoscenze esterne e minimizzando il rischio di allucinazioni.



6.3. Ottimizzazione NLP: Risoluzione delle Allucinazioni da Identificativo

Durante le fasi di collaudo del sistema di chat contestuale per documenti complessi (PDF e file Markdown), è emerso un problema noto nell'ambito del *Prompt Engineering*: il degrado dell'attenzione dell'LLM causato da stringhe alfanumeriche complesse. Inizialmente, i *chunk* recuperati venivano passati al modello accompagnati dall'ID univoco dell'articolo. La presenza massiccia di queste stringhe incomprensibili nel contesto causava confusione nell'Intelligenza Artificiale, portandola a ignorare il contenuto utile o a non generare alcuna risposta. Per risolvere questa criticità

architetturale, l'endpoint è stato implementato affinché esegua uno step intermedio di “risoluzione dell'identificativo”: prima di invocare Azure OpenAI, il backend interroga Cosmos DB per mappare dinamicamente ogni `article_id` con il rispettivo titolo in linguaggio naturale. Sostituendo l'ID crittografico con il titolo editoriale nel prompt, la comprensione del contesto da parte del modello è drasticamente migliorata, permettendo all'agente conversazionale di citare le fonti in modo organico ed elegante.

6.4. Tracciabilità delle Fonti e Interfacce Utente

Un vincolo fondamentale del progetto imponeva di garantire sempre all'utente la possibilità di risalire, partendo da una risposta generata, al documento originale. A livello logico, questo requisito è stato soddisfatto strutturando gli endpoint RAG affinché restituiscano un payload JSON ibrido: un campo contenente la stringa generata dall'IA e un array strutturato contenente le fonti esatte (i chunk e i relativi metadati) utilizzate per formularla. Lato frontend, questa separazione ha permesso di implementare due interfacce distinte e altamente professionali:

- **Ricerca Globale:** una schermata RAG dedicata, dove la risposta dell'IA è accompagnata da *Card* navigabili che mostrano gli articoli sorgente da cui ha attinto le informazioni.
- **Chat Contestuale (Locale):** un pannello a comparsa (*Drawer* laterale) integrato nella vista di dettaglio del singolo articolo, che permette all'utente loggato di “dialogare” in tempo reale con lo specifico documento aperto, isolando la conversazione dal resto dell'archivio.

7. Esperienza Utente e Interfaccia Grafica

I capitoli precedenti hanno descritto l'infrastruttura e la logica applicativa; questo capitolo affronta il livello con cui l'utente interagisce concretamente. La scelta di dedicare al frontend una trattazione autonoma nasce da una considerazione progettuale: in un sistema RAG la qualità percepita non dipende solo dall'accuratezza del retrieval, ma anche dalla capacità dell'interfaccia di rendere comprensibile all'utente la provenienza di una risposta e quali azioni sono consentite in un determinato stato dell'applicazione.

7.1. Impostazione dell'Interfaccia e Design System

L'applicazione è realizzata in Flutter e adotta le linee guida Material 3. L'intera interfaccia è costruita su una tavolozza cromatica ridotta, applicata secondo la regola 60-30-10: un grigio chiarissimo come sfondo dominante, un marrone come colore istituzionale di barre e intestazioni e un arancione riservato esclusivamente agli elementi interattivi e agli stati attivi. La conseguenza pratica è che la presenza dell'arancione in una schermata segnala sempre un'azione possibile, il che riduce il carico cognitivo richiesto per orientarsi.

La navigazione è organizzata attorno ad un menù laterale (Drawer) condiviso per tutte le schermate, che espone le sole voci coerenti con lo stato della sessione: se l'utente non è autenticato il menù propone l'accesso; mentre, le voci relative al caricamento, alla cronologia e al logout sono utilizzabili solo se la sessione è attiva. Il controllo non si riduce solo all'estetica: come descritto nel capitolo sulla sicurezza, gli endpoint corrispondenti sono protetti anche lato server; inoltre, l'interfaccia si limita a evitare che l'utente intraprenda azioni che verrebbero comunque respinte.

7.2. Accesso e Registrazione

L'autenticazione avviene all'interno dell'applicazione, evitando reindirizzamenti verso le pagine predefinite di Keycloak: è questa la ragione per cui, come discusso nel capitolo sulla gestione delle identità, è stato abilitato il flusso Direct Access Grants. Accesso e registrazione condividono un unico pannello, in cui una banda diagonale animata scorre lateralmente per passare da una modalità all'altra, mantenendo la continuità visiva ed evitando il cambio di schermata.

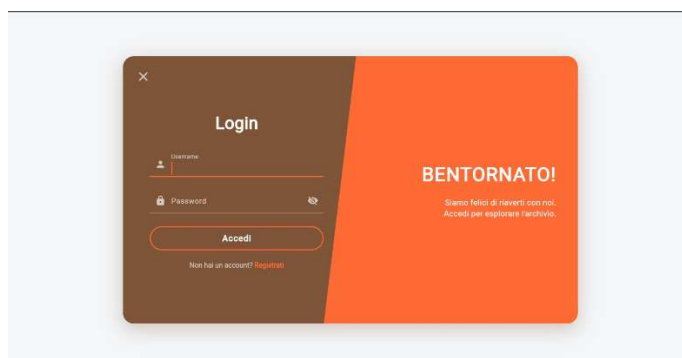


Figura 7.1 Schermata di accesso

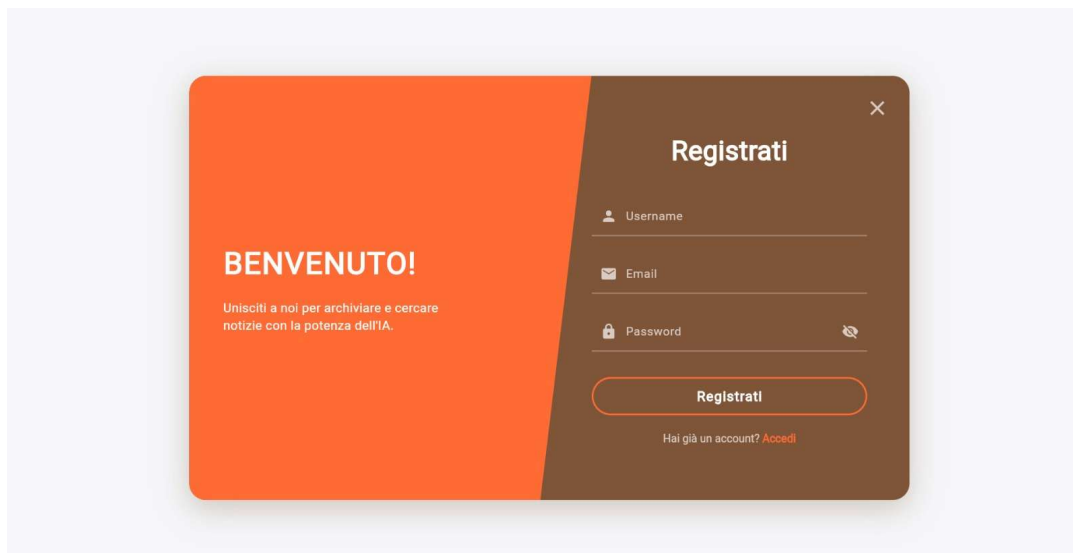


Figura 7.2 Schermata di registrazione

Al termine di un accesso a buon fine il client memorizza la coppia di token restituita da Keycloak insieme alle rispettive scadenze, informazioni che consentono di rinnovare l'Access Token prima che scada anziché attendere il rifiuto di una richiesta. Il rinnovo è quindi invisibile all'utente, che percepisce una sessione continua; la schermata di accesso ricompare solo quando il Refresh Token non è più spendibile e la sessione è effettivamente terminata.

7.3 Dashboard ed Esplorazione dell'Archivio

La schermata principale presenta l'archivio come una griglia di card, ciascuna delle quali riporta l'immagine di copertina, le categorie associate, il titolo e l'autore. Il caricamento adotta lo scorrimento infinito descritto nel capitolo precedente: al raggiungimento della parte finale della lista il client richiede la pagina successiva tramite i parametri skip e limit, senza che l'utente debba interagire con controlli di paginazione.

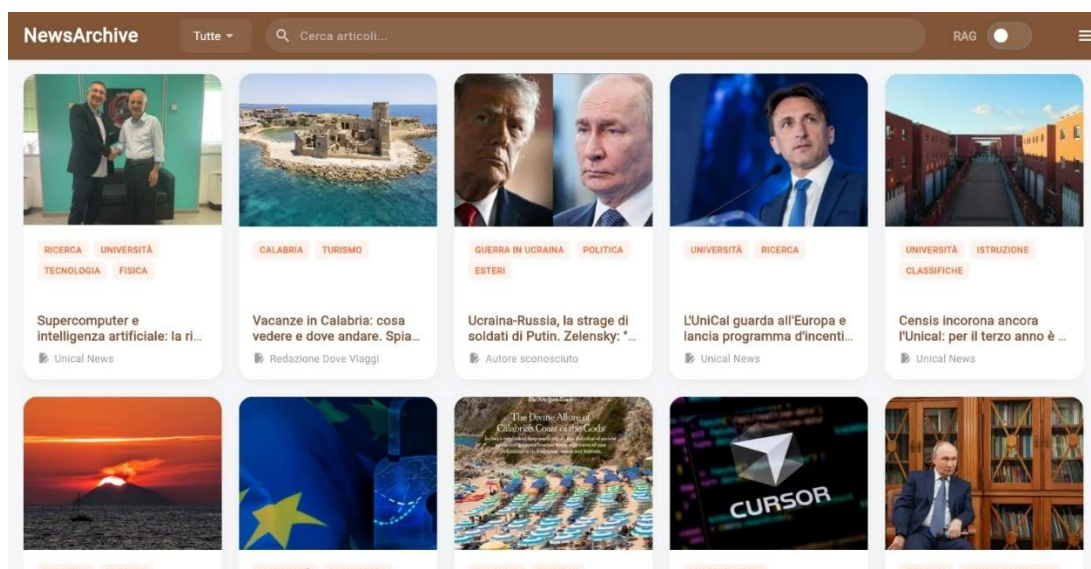


Figura 7.3 Dashboard principale con la griglia degli articoli

La barra superiore concentra i tre strumenti di esplorazione: il filtro per categoria, il campo di ricerca e l'interruttore che commuta tra ricerca testuale e ricerca RAG. La distinzione è resa esplicita all'utente perché le due modalità hanno costi e requisiti diversi: la ricerca testuale è libera e accessibile anche senza account, mentre la modalità RAG richiede l'autenticazione.

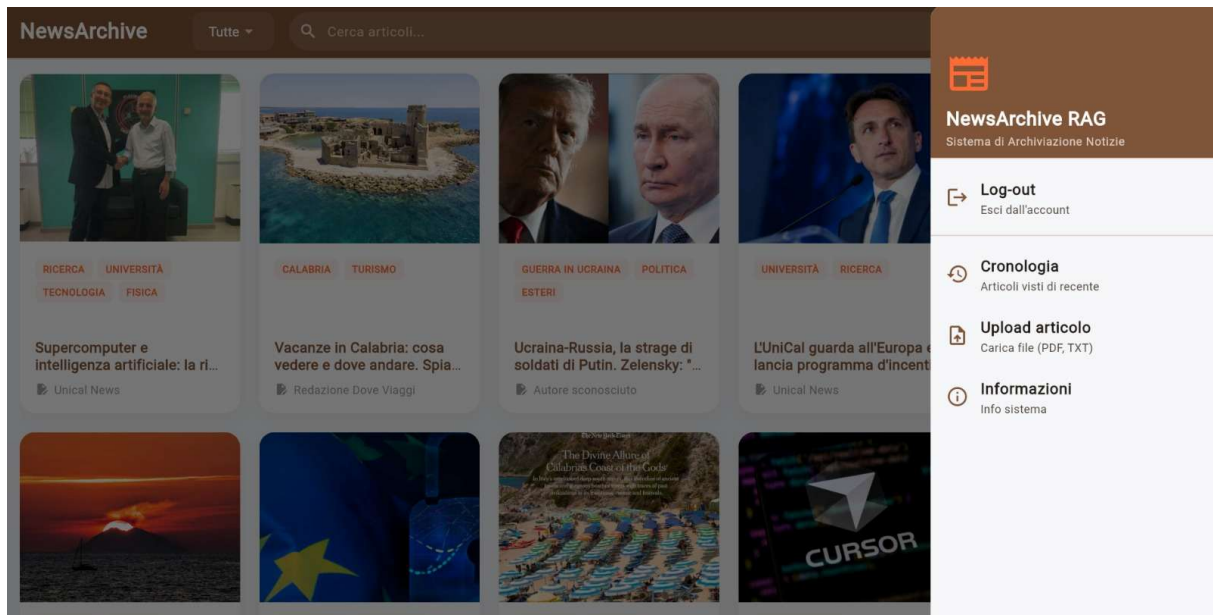


Figura 7.4 Menu laterale a sessione attiva

7.4 Caricamento di un Articolo

La schermata di caricamento è divisa in due sezioni distinte, che rispecchiano la natura dei dati richiesti. La prima riguarda i file: il documento e l'immagine di copertina (opzione facoltativa). I formati ammessi sono indicati direttamente nell'etichetta del campo, così da rendere evidente il vincolo prima della selezione anziché comunicarlo con un errore successivo.

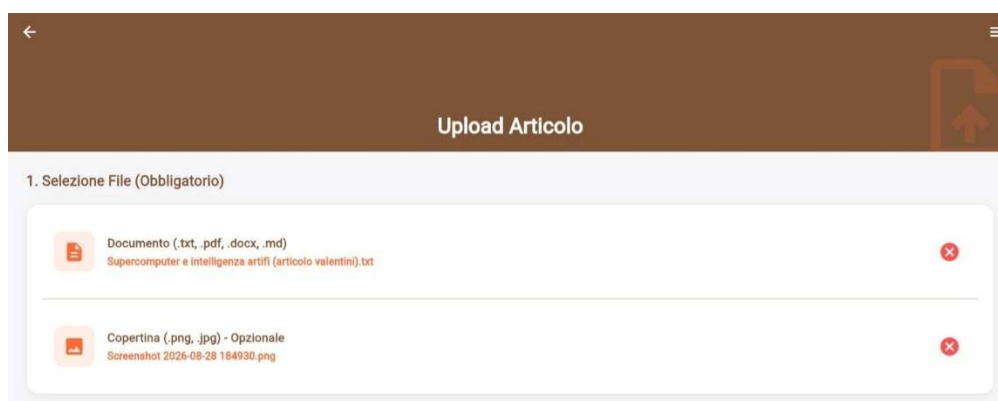


Figura 7.5 Selezione del documento e della copertina

La seconda sezione raccoglie i metadati manuali previsti dalle specifiche: titolo, autore, categorie, descrizione e tag. Categorie e tag non sono campi di testo liberi ma insiemi di elementi rimovibili, una scelta che rende visibile in ogni momento l'elenco di quanto già inserito e consente di correggere un singolo elemento senza riscrivere l'intero campo. I metadati generati automaticamente non compaiono in questa schermata: vengono prodotti dalla pipeline descritta nel capitolo sull'ingestion e mostrati solo nella scheda finale dell'articolo.

2. Dati dell'Articolo (Metadati)

Titolo dell'articolo
T Supercomputer e intelligenza artificiale: la ricerca dell'UniCal punta alla fisica del futuro

Autore
Unical News

Ricerca X Università X Tecnologia X Fisica X

Aggiungi una categoria
Seleziona una categoria...

Descrizione breve
L'Università della Calabria si è aggiudicata una parte di un finanziamento europeo da 8 milioni di euro per il progetto PLASMA-PEPSC 2. L'Ateneo co-guiderà l'integrazione di supercomputer exascale e algoritmi di intelligenza artificiale per studiare la fisica computazionale dei plasmi, supportando ricerche sulla fusione nucleare e lo Space Weather.

Unical X PLASMA-PEPSC 2 X Supercomputer X Intelligenza Artificiale X Fisica del Plasma X Francesco Valentini X Domenico Talia X

Scrivi un Tag e premi Invio

CARICA ARTICOLO

Figura 7.6 Compilazione dei metadati manuali

Al termine del caricamento l'utente riceve un riscontro esplicito sull'esito. I due controlli di deduplicazione descritti in precedenza possono infatti respingere la richiesta, facendo sì che compaia un messaggio di errore.

7.5 Gestione delle Categorie

La gestione delle categorie attraversa tutti e tre i livelli dell'architettura ed è stata progettata attorno ad un vincolo preciso: l'insieme delle categorie non è definito a priori in fase di sviluppo, ma cresce insieme all'archivio, poiché è l'utente stesso a introdurne di nuove nel momento in cui carica un articolo che non rientra in quelle esistenti.

- **Persistenza dedicata:** le categorie non vengono ricavate scandendo gli articoli, ma risiedono in un container Cosmos DB dedicato. La motivazione è di costo: ricostruire l'elenco a partire dagli articoli richiederebbe una query cross-partition sull'intera collezione ogni volta che il client deve popolare un menu, mentre il container delle categorie resta piccolo e la sua lettura economica anche a fronte di un archivio in crescita.
- **Deduplicazione case-insensitive:** ogni categoria è salvata con un identificativo pari al nome normalizzato in minuscolo e con un campo separato che conserva la forma originale scelta dall'utente. Poiché in Cosmos DB l'identificativo è univoco, ad esempio: le scritture “Sport”, “sport” e “SPORT” convergono necessariamente

sullo stesso documento e l'archivio non si popola di varianti tipografiche della medesima categoria.

- **Creazione implicita:** non esiste un endpoint amministrativo per la creazione delle categorie. La funzione `add_category_if_not_exists` viene invocata dall'endpoint di upload per ciascuna categoria associata all'articolo; se il documento non esiste viene creato, altrimenti l'operazione termina senza effetti. Il controllo di esistenza è realizzato con una lettura puntuale sull'identificativo, con chiave di partizione coincidente con esso, anziché con una query: è l'operazione meno costosa offerta dal servizio, il che rende sostenibile invocarla a ogni caricamento anche quando la categoria è già presente.
- **Esposizione al client:** l'endpoint `GET /articles /categories` restituisce l'elenco completo ordinato alfabeticamente, così che il frontend non debba mai ricostruirlo autonomamente.

Sul lato dell'interfaccia, la selezione è affidata a un componente riutilizzabile che apre un pannello a comparsa dal basso, con le categorie raggruppate in sezioni alfabetiche a intestazione persistente, secondo una modalità di consultazione analoga a quella di una rubrica. Il medesimo componente è impiegato in tre punti diversi dell'applicazione, con comportamenti leggermente differenti: nel filtro della dashboard espone anche la voce che rimuove il filtro, nel form di caricamento e nella finestra di modifica espone invece il pulsante che consente di introdurre una categoria non ancora presente.

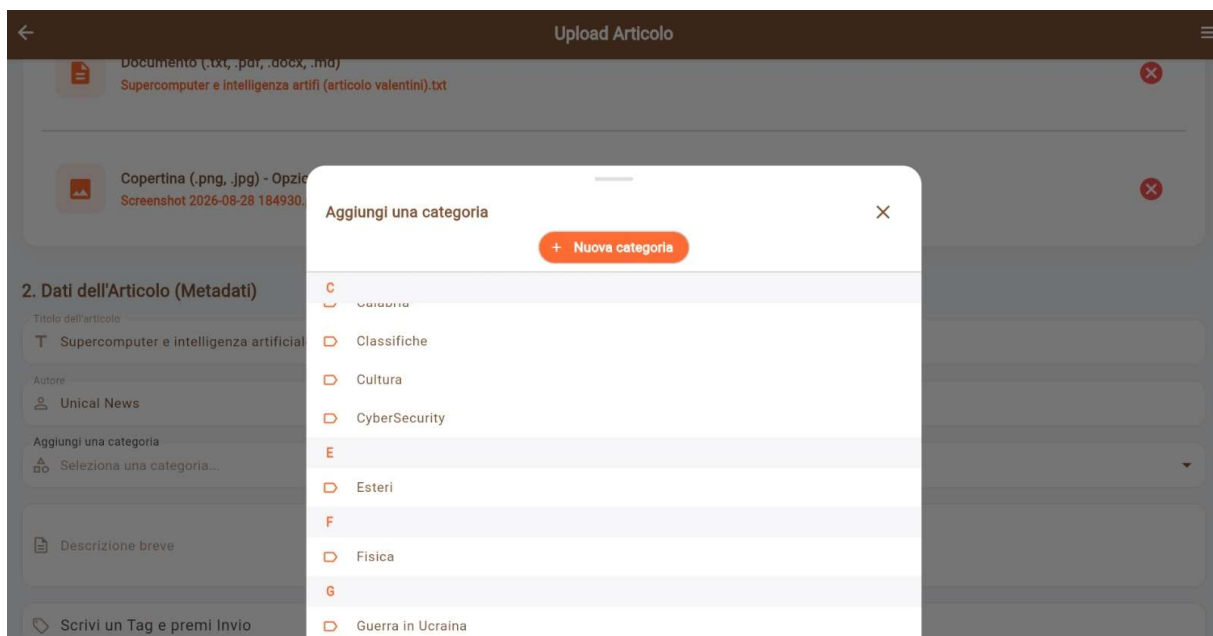


Figura 7.7 Pannello di selezione delle categorie

Quando l'utente sceglie di aggiungere una categoria nuova, il componente non la crea immediatamente: restituisce alla schermata chiamante un valore convenzionale che ne segnala l'intenzione, e il nome digitato resta un elemento locale del form finché l'articolo non viene effettivamente caricato. La categoria viene resa persistente soltanto contestualmente all'upload, quando il backend la registra insieme all'articolo. Questa

scelta evita che tentativi di caricamento abbandonati, o respinti dai controlli di deduplicazione, lascino nell'archivio categorie prive di articoli associati.

Il filtro della dashboard, infine, non opera lato client sui risultati già scaricati, ma viene trasmesso al backend come parametro della richiesta e tradotto in una condizione `ARRAY_CONTAINS` sul campo delle categorie dell'articolo, preservando così la coerenza con la paginazione.

7.6 Cronologia Personale e Operazioni CRUD

La cronologia raccoglie i soli articoli caricati dall'utente autenticato. L'elenco non viene filtrato sul client: il backend estrae l'identificativo dell'utente dal token e restringe la query, coerentemente con la separazione delle responsabilità descritta nel capitolo sulla sicurezza. Da questa schermata sono raggiungibili le operazioni di modifica ed eliminazione, che completano il paradigma CRUD tra i requisiti aggiuntivi del progetto.



Figura 7.8 Cronologia personale dell'utente

La modifica avviene in una finestra di dialogo che ripropone i soli metadati manuali, lasciando immutati il documento originale e i metadati generati dall'intelligenza artificiale. L'eliminazione, in quanto irreversibile, richiede una conferma esplicita e comporta lato server la rimozione coordinata del documento, dei suoi frammenti e dei relativi vettori dall'indice di ricerca.



Figura 7.9 Finestra di modifica dei metadati

7.7 Scheda dell'Articolo e Chat Contestuale

La scheda di dettaglio distingue visivamente le due nature dei metadati: quelli inseriti manualmente dall'utente e quelli prodotti dall'analisi automatica. La separazione è deliberata, perché consente al lettore di valutare quali informazioni sono state dichiarate e quali invece inferite da un modello linguistico, ed è coerente con il requisito di tracciabilità che attraversa l'intero progetto.



Figura 7.10 Scheda di dettaglio di un articolo

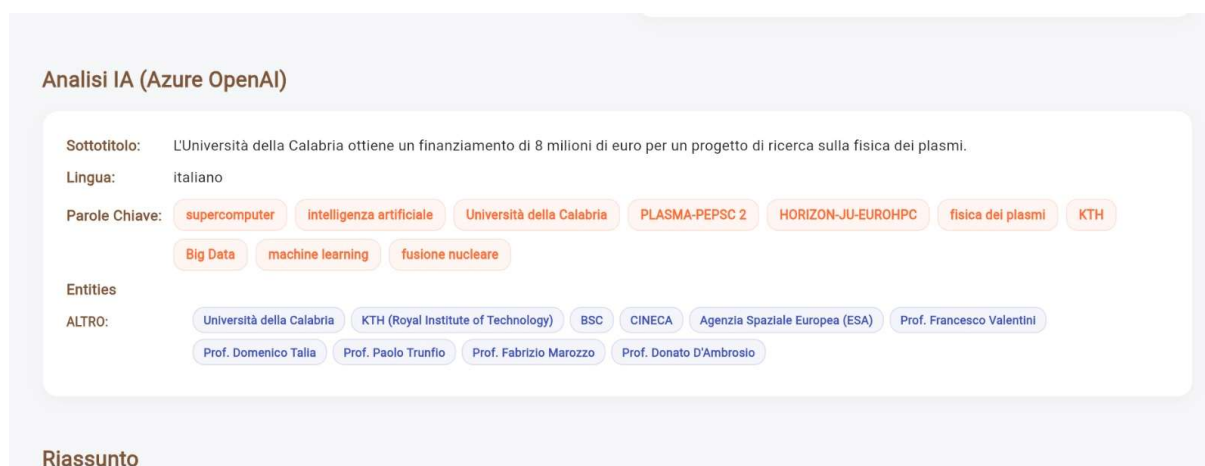


Figura 7.11 Metadati generati automaticamente

In fondo alla scheda sono disponibili il download del documento originale e l'esportazione dei metadati, mentre un pulsante persistente apre il pannello di chat contestuale descritto nel capitolo precedente, che circonda la conversazione all'articolo in lettura.



Figura 7.12 Riassunto generato e funzioni di esportazione

7.8 Ricerca RAG e Tracciabilità delle Fonti

Attivando la modalità RAG, la risposta generata dal modello viene presentata in un riquadro distinto, graficamente separato dal resto dei contenuti per non lasciare dubbi sulla sua origine automatica. Immediatamente sotto compaiono, con l'intestazione "Articoli Consultati", le card degli articoli da cui provengono i frammenti effettivamente utilizzati: è la traduzione in interfaccia del payload ibrido descritto nel capitolo precedente, e soddisfa il requisito di poter sempre risalire dalla risposta ai documenti originali.

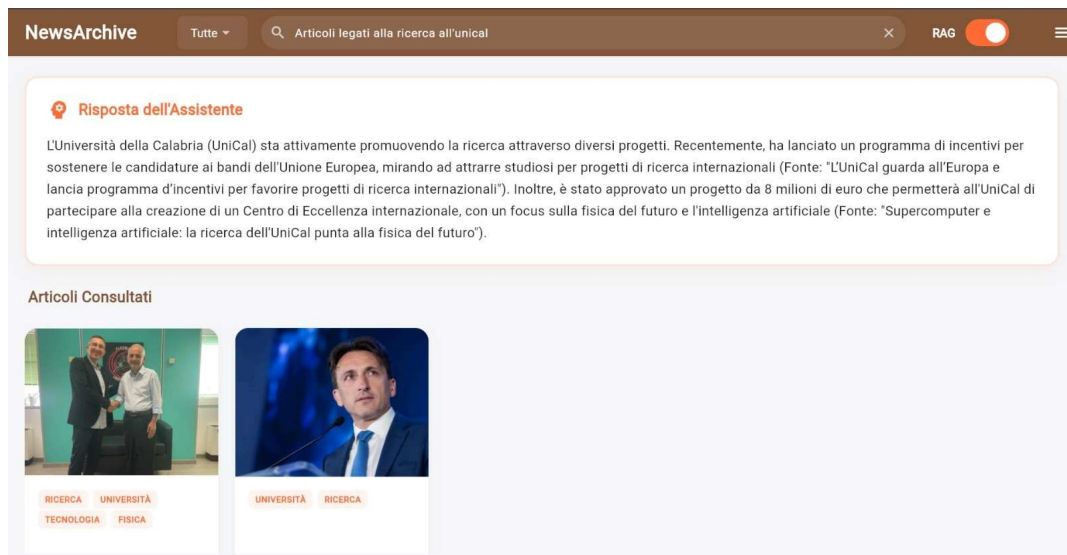


Figura 7.13 Risposta RAG con gli articoli consultati

Le fonti sono presentate nell'ordine di rilevanza restituito dal motore di ricerca e sono navigabili: selezionando una card si accede alla scheda completa dell'articolo, chiudendo il percorso che va dalla domanda in linguaggio naturale al documento archiviato.

7.9 Informazioni di Sistema

Una schermata dedicata riepiloga le tecnologie impiegate e i riferimenti del progetto. Pur non rispondendo a un requisito funzionale, rende l'architettura leggibile dall'interno dell'applicazione stessa, senza richiedere la consultazione della presente relazione.



Figura 7.14 Schermata informativa sul sistema

8. Ciclo di Vita dell'Applicazione e Virtualizzazione

Il deployment e la gestione del ciclo di vita dell'applicazione sono stati progettati per rispettare rigorosamente il vincolo di progetto che impone l'utilizzo delle soluzioni di calcolo e virtualizzazione messe a disposizione da Microsoft Azure. Parallelamente, è stata definita una rigorosa strategia di ottimizzazione dei costi per far fronte ai limiti imposti dalla sottoscrizione Azure for Students (budget di 100 € — verificare valuta e importo).

8.1. Containerizzazione e Strategia di Virtualizzazione

In linea con i paradigmi Cloud Native, gli strati applicativi non erogati come servizi PaaS gestiti sono stati isolati e virtualizzati tramite container Docker. Nello specifico, il backend (FastAPI) e l'Identity Provider (Keycloak) sono stati pacchettizzati in immagini Docker indipendenti. Questa astrazione a livello di sistema operativo permette il deployment su servizi di calcolo Azure, quali Azure Virtual Machines (IaaS) o Azure Container Instances (CaaS); quest'ultima opzione, in particolare, consente di eseguire i container in modalità serverless, eliminando l'overhead di gestione del sistema operativo sottostante e garantendo un indirizzo IP pubblico immediato. Il client frontend, sviluppato in Flutter (Dart), non necessita invece di un servizio di hosting web dedicato su Azure: viene eseguito interamente lato client (desktop o mobile) e si interfaccia con le risorse cloud invocando l'indirizzo IP pubblico del backend tramite chiamate HTTP REST.

8.2. Mappatura dei Ruoli Architeturali

L'architettura distribuita del sistema riflette concettualmente i pattern classici del cloud computing basati sulla separazione delle responsabilità computazionali:

- **Web Role:** l'istanza FastAPI funge da interfaccia pubblica, ricevendo le richieste REST dal client, validando i token di sicurezza e orchestrando le pipeline RAG.
- **Worker Role (Logica di Background):** le operazioni I/O-bound e computazionalmente intensive — parsing dei file, invocazione di Azure OpenAI per la generazione dei metadati, chunking testuale — sono gestite in modo nativamente asincrono all'interno del backend tramite asyncio, evitando il blocco dei thread principali del server. L'integrazione di questi ruoli computazionali con lo strato dati aderisce al paradigma cloud che prevede l'assenza di database relazionali (Zero SQL), delegando l'archiviazione di documenti e vettori rispettivamente a Blob Storage e Cosmos DB NoSQL.

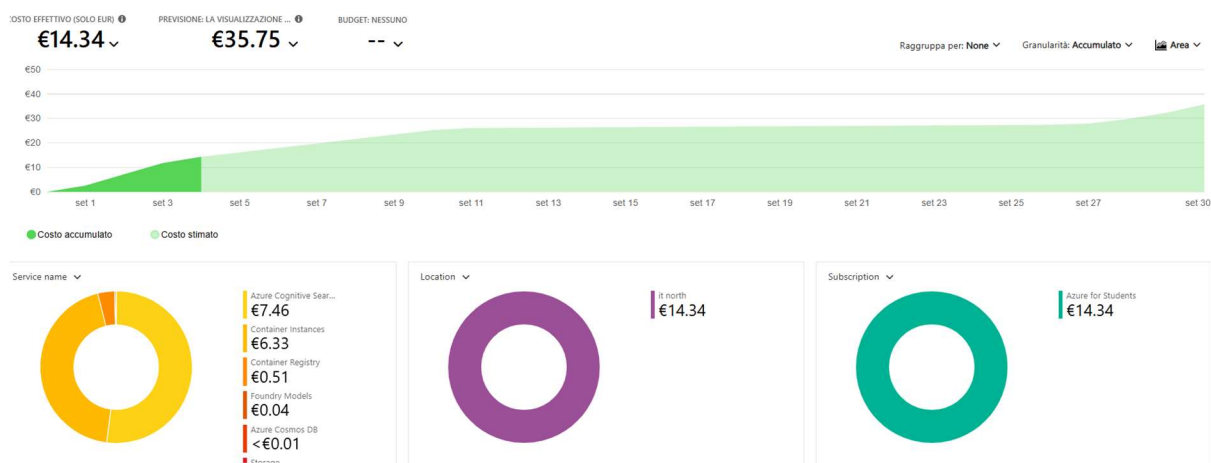
8.3. Ambienti di Sviluppo e Gestione del Budget

Per preservare il limitato budget studentesco durante le lunghe fasi di sviluppo e test end-to-end, è stata ingegnerizzata un'architettura ibrida basata su emulatori locali e tecniche di mocking, definendo un ambiente di collaudo a costo zero.

- **Emulazione dello Storage:** le letture e scritture dei file fisici sono state reindirizzate verso Azurite, l'emulatore locale ufficiale di Microsoft per Blob Storage.
- **Emulazione del Database:** le operazioni sui metadati (inserimento, deduplicazione tramite controllo dei titoli, query per la dashboard) sono state convogliate sull'Azure Cosmos DB Emulator locale.
- **Mocking dell'Intelligenza Artificiale:** tramite la variabile d'ambiente TEST_MODE=True, il backend è stato istruito a bypassare le costose chiamate di rete verso Azure OpenAI e Azure AI Search, restituendo vettori e metadati fittizi precompilati.

8.4. Strategia di Deployment per la Release Finale

In vista della demo finale, la gestione dell'infrastruttura ha richiesto un bilanciamento tra l'ottimizzazione estrema dei costi e la riduzione del rischio operativo (risk management). L'architettura PaaS di Azure lega indissolubilmente la persistenza dei dati all'esistenza dell'istanza di calcolo; pertanto, la distruzione del servizio Azure AI Search comporterebbe la perdita irreparabile dell'intero indice vettoriale. Attraverso un'analisi dei consumi previsti, si è stimato che il costo di mantenimento dell'infrastruttura *always-on* per i 10 giorni antecedenti la demo ammonti a circa [30] €, portando la spesa totale del progetto a circa [70] € — ampiamente entro il limite di [100] € fornito dalla sottoscrizione. Di conseguenza, al fine di garantire l'affidabilità assoluta del sistema durante la dimostrazione e prevenire la necessità di eseguire script di re-indicizzazione dell'ultimo minuto, si è optato per mantenere attiva l'infrastruttura di produzione. Come contromisura di sicurezza contro eventuali picchi anomali di consumo, è stato configurato un Azure Budget Alert con soglia di allarme impostata a [50] €.



9. Registro delle Interazioni con IA generativa

Gli strumenti di Intelligenza Artificiale Generativa non sono stati impiegati per la generazione di codice sorgente “alla cieca”, ma hanno operato prevalentemente come *Code Reviewer* esperti e consulenti architetturali per la validazione di scelte ingegneristiche complesse. Nello specifico, l'ausilio dell'IA è stato fondamentale in tre ambiti principali:

- **Decisioni Architetture (ADR):** analisi comparativa di protocolli di sicurezza e pattern di distribuzione cloud.
- **Code Review e Debugging:** identificazione di colli di bottiglia (deadlock asincroni in Python) e controllo di conformità del codice rispetto alle API ufficiali dei servizi Microsoft.
- **Refactoring e Terminologia:** l'Intelligenza Artificiale è stata utilizzata attivamente per migliorare la leggibilità del codice e completare la stesura della documentazione e della presente relazione, affinando il testo tramite l'adozione di un registro linguistico e di una terminologia tecnica appropriata al contesto dell'ingegneria del software.

L'utilizzo è avvenuto in due fasi distinte: una prima fase di contestualizzazione, in cui è stato fornito al modello il contesto progettuale e i requisiti specifici, funzionale a delineare in modo puntuale gli obiettivi da perseguire e le strategie di refactoring più adeguate; una seconda fase operativa, in cui tali indicazioni sono state applicate concretamente al codice sorgente e ai test, tramite prompt quali

Di seguito si riportano i log dettagliati delle interazioni, suddivisi per livello architetturale.

9.1 Infrastruttura Cloud (Azure) e Infrastructure as Code

Domanda 1: Agisci come un Senior Cloud/Software Engineer. Voglio che tu mi spieghi come implementare un file Bicep per il deploy di risorse su Azure. La tua spiegazione deve essere pratica, strutturata e orientata al problem-solving.

Per favore, includi i seguenti punti nella tua risposta:

1. **Concetti Core:** una sintesi rapida della sintassi essenziale (parametri, variabili, risorse e output).
2. **Modularità e Struttura:** spiega come strutturare il codice in moduli Bicep separati (es. `main.bicep` che richiama sotto moduli) per favorire la manutenibilità e la riusabilità del codice infrastrutturale.
3. **Esempio Pratico:** scrivi un file Bicep di esempio (o una struttura a moduli) che effettui il provisioning di un'architettura backend tipica (ad esempio, un account

Azure Storage/Blob e un'istanza Cosmos DB). Aggiungi commenti al codice per spiegare le scelte implementative.

4. **Best Practice e Sicurezza:** come gestire i parametri sensibili (secret) e quali sono le convenzioni raccomandate per il naming delle risorse.
5. **Deploy e CI/CD:** i comandi Azure CLI (o Azure PowerShell) necessari per validare (what-if) ed eseguire il deploy del file, includendo un breve accenno a come il processo si integra in una pipeline automatizzata.

Usa un tono professionale, tecnico e conciso. Evita introduzioni prolisse e vai dritto al codice e all'architettura.

Modello usato: Google Gemini

Risposta: Spiegazione strutturata dell'approccio Infrastructure as Code (IaC) tramite Bicep, il DSL Microsoft che astrae i template ARM (JSON) semplificandone stesura e manutenzione. La risposta ha coperto cinque aree:

1. **Concetti core:** struttura di un file Bicep in quattro sezioni — param (input al deploy, es. ambiente dev/prod), var (valori calcolati a runtime, es. naming policy), resource (servizi Azure da istanziare) e output (dati esposti a fine deploy, utili quando il file è richiamato come modulo).
2. **Modularità:** indicazione di suddividere l'infrastruttura complessa in file separati (es. storage.bicep, cosmos.bicep) richiamati dal main.bicep tramite la keyword module, invece di concentrare tutto in un unico file.
3. **Esempio pratico:** generazione di un main.bicep completo per un backend tipico, comprendente il provisioning di uno Storage Account (Blob Storage, SKU Standard_LRS, container dedicato ai documenti) e di un'istanza Cosmos DB (GlobalDocumentDB, consistency level Session), con naming delle risorse reso univoco tramite uniqueString(resourceGroup().id) e output degli endpoint (storageEndpoint, cosmosDbEndpoint) per l'integrazione con altri moduli.
4. **Pattern ingegneristici:** approfondimento su tre proprietà chiave di Bicep — **idempotenza** (deploy ripetuti non duplicano le risorse, ma verificano lo stato corrente rispetto al codice dichiarato), **gestione automatica delle dipendenze** (Bicep costruisce un DAG implicito tramite riferimenti tra risorse, es. parent o richiami diretti, parallelizzando il deploy dove possibile), e **modularità** per la scalabilità del codice infrastrutturale.
5. **Best practice, sicurezza e deploy:** cenni su gestione dei parametri sensibili e convenzioni di naming, oltre ai comandi Azure CLI per la validazione (what-if) e l'esecuzione del deploy, con riferimento alla loro integrazione in pipeline CI/CD automatizzate.

Domanda 2: Verifica la correttezza dei file Bicep. Aggiungi commenti esplicativi e genera il README con istruzioni per Azure CLI.

Modello usato: Claude 3.5 Sonnet

Risposta: Validazione strutturale degli script Bicep con inserimento di commenti esplicativi sulle singole risorse cloud. Generazione del file README.md contenente i comandi operativi (az login, az group create, az deployment group create) per il provisioning automatico dell'ambiente. Questo documento si può trovare all'interno della repository git hub nella cartella dedicata ad Azure

Struttura

```
Azure/
├─ main.bicep          # Entry point: crea RG + chiama tutti i moduli
├─ parameters.json     # Parametri configurabili (RG name, location, prefix)
├─ modules/
│   ├── storage.bicep   # Blob Storage + container articles-raw
│   ├── cosmos.bicep   # Cosmos DB Serverless + database + containers
│   ├── search.bicep    # Azure AI Search (tier Basic)
│   ├── openai.bicep    # Azure OpenAI Service – solo account base (*)
│   └─ acr.bicep        # Azure Container Registry (Basic)
```

(*) **Nota — Policy Azure for Students (Università della Calabria)** La sottoscrizione Azure for Students dell'UniCal impone l'uso esclusivo della regione **italynorth** e blocca il provisioning automatico verso regioni esterne. Per questa ragione il modulo `openai.bicep` crea **soltanto l'account** del servizio (`Microsoft.CognitiveServices/accounts`); i **deployment dei modelli** (`text-embedding-ada-002` e `gpt-4o-mini`) vengono distribuiti manualmente tramite **Azure AI Studio / Portale Azure** dopo il completamento del deploy IaC.

Prerequisiti

1. **Azure CLI** installata e configurata:

```
az login
az account set --subscription <SUBSCRIPTION_ID>
```

Domanda 3: Adatta il modulo Bicep OpenAI ai vincoli Azure for Students UniCal (italynorth obbligatorio). Solo account base; deployment modelli via Azure AI Studio.

Modello usato: Claude 3.5 Sonnet

Risposta: Refactoring architetturale del file `openai.bicep`. Sono stati rimossi i blocchi di deployment automatico dei modelli. Sono stati di conseguenza aggiornati i file `main.bicep`, `parameters.json` e la documentazione del progetto, conformando l'infrastruttura alla rigida policy della sottoscrizione universitaria. Inoltre, è stata

generata la procedura per un deploy dei modelli specificata sempre nel file README.md.

9.2 Sviluppo Backend e Logia Applicativa

In questa specifica parte i modelli sono stati interrogati prevalentemente per ottemperare alle best practice il più possibile prima di scrivere righe di codice, favorendo la modularità e tutte le proprietà descritte negli altri capitoli.

Domanda 4: Dopo le delucidazioni architetturali esplicitate nel corso della chat spiegami la best practice per la generazione di un file .env e creami il file backend/.env.example mappando le variabili d'ambiente secondo la struttura Bicep.

Modello usato: Claude 3.5 Sonnet

Risposta: Validazione della sintassi delle chiavi d'infrastruttura. Verifica della corrispondenza esatta tra le variabili fornite e le attese dei client SDK Python (Storage, Cosmos, Search, OpenAI). Generazione del file .env.example privo di credenziali e verifica delle regole di esclusione nel file .gitignore per prevenire fughe di dati.

Domanda 5: Code review index_chunk_to_ai_search: uso di zip(), enumerate(), serializzazione Pydantic, client asincrono, blocco TEST_MODE.

Modello usato: Claude 3.5 Sonnet

Risposta: Il codice è stato scritto interamente dallo studente; l'IA ha svolto pura funzione di review. Sono stati corretti 4 bug infrastrutturali: (1) Sostituzione dell'import sincrono con quello asincrono (azure.search.documents.aio.SearchClient) per prevenire il blocco del server FastAPI; (2) Fix della chiave errata in AZURE_SEARCH_ADMIN_KEY; (3) Aggiunta del blocco logico TEST_MODE mancante; (4) Correzione dell'indentazione del blocco funzionale.

9.3 Gestione delle Identità e Sicurezza

Domanda 6: Valuta JWT custom vs MSAL per il progetto.

Modello usato: Gemini 1.5 Pro

Risposta: Analisi comparativa sui protocolli di sicurezza. Scelta motivata verso l'adozione di Keycloak e del protocollo OAuth2/OIDC, scartando l'approccio JWT manuale per rispettare la regola d'oro della sicurezza informatica “Don't roll your own crypto”, aggirando al contempo l'impossibilità di creare un tenant Azure Entra ID B2C.

Domanda 7: Ragiona come un web Security expert professionista e assicurati che il codice di questa vecchia configurazione sia adattabile al progetto in questione, in caso proponimi delle modifiche da effettuare.

Modello usato: Gemini 1.5 Pro

Risposta: Code review e analisi di vulnerabilità sulla base di codice preesistente.

L'Intelligenza Artificiale ha guidato il refactoring necessario per adattare il codice ai nuovi paradigmi architetturali, identificando e correggendo criticità di sicurezza (come la rimozione dell'invio delle password in chiaro tramite query string URL nel client frontend). Sono state inoltre proposte e implementate le modifiche al backend FastAPI, integrando il middleware HTTPBearer per il download asincrono delle chiavi pubbliche e la validazione crittografica dei token JWT, rendendo la configurazione legacy pienamente compatibile e sicura rispetto al nuovo Identity Provider (Keycloak).

Domanda 8: Si può automatizzare il refresh token di Keycloak? (Richiesta effettuata a valle dell'analisi dei problemi di blocco della User Experience durante la scadenza della sessione).

Modello usato: Gemini 1.5 Pro

Risposta: Suggerimento implementativo di altissimo livello. L'IA ha guidato la creazione di un endpoint FastAPI dedicato (/refresh) e, lato frontend, ha fornito il codice per un ApiHelper intelligente (Wrapper HTTP) capace di intercettare gli errori HTTP 401, scambiare asincronamente il *refresh token*, e rieseguire la chiamata HTTP originale in modo totalmente trasparente per l'utente, risolvendo i blocchi dell'interfaccia.

9.3 Sviluppo Frontend e User Experience

Domanda: Ragiona come un web Designer professionista e analizza il seguente codice e dimmi se posso apportare le seguenti modifiche:

- 1) centralizzazione e globalizzazione di variabili di istanza;
- 2) aggiungimi uno switcher per la modalità Rag;
- 3) e commentami dove è localizzato il menù a tendina;
- 4) inoltre, consigliami quali parti del frontend posso modularizzare;

Modello usato: Gemini 1.5 Pro

Risposta: Code review focalizzata sulla UI/UX e sull'architettura del frontend client in Flutter. L'Intelligenza Artificiale ha applicato la regola di design “60-30-10” centralizzando le variabili di stile (colori di sfondo, primari e di accento) a livello globale per garantire coerenza visiva. È stato integrato con successo uno switch interattivo (Toggle) per attivare la modalità RAG, posizionandolo strategicamente nell'AppBar. Infine, il modello ha analizzato il posizionamento del menù a tendina (End Drawer) e fornito direttive specifiche sulla modularizzazione del codice: ha guidato l'estrazione del pannello laterale in un widget autonomo (menu_laterale.dart) per prevenire la ridondanza e la duplicazione della logica di stato nelle diverse schermate dell'applicativo.

Domanda 10: Data la mia scelta progettuale di non voler uscire dalla user Experience spiegami come implementare la transizione di login in video.

Modello usato: Gemini 1.5 Pro

Risposta: Trasposizione complessa di un'animazione web (HTML/CSS) in widget Flutter nativi (utilizzo combinato di Stack, AnimatedPositioned e AnimatedOpacity).

Adattamento rigoroso al *Design System* del progetto e risoluzione di bug relativi all'overlay dei campi di testo, rendendo il form pronto per interfacciarsi con il flusso API *Direct Access Grants* di Keycloak senza mai abbandonare l'applicativo Flutter.

Domanda 11: Implementazione della gestione categorie articoli (backend Cosmos DB + frontend Flutter con pannello a comparsa e sezioni alfabetiche), debug di errori di compilazione Dart e di configurazione Cosmos DB/Bicep, persistenza del realm Keycloak su Azure Container Instances tramite Azure File Share, e infine implementazione del refresh automatico dei token JWT (access/refresh token Keycloak) lato app Flutter.

Modello usato: Claude Sonnet 5

Risposta: Sono stati creati/modificati: un container Cosmos DB dedicato “categories” con relative funzioni backend (FastAPI) per lettura, creazione e backfill delle categorie; un widget Flutter riutilizzabile con bottom sheet a sezioni alfabetiche “sticky” per la selezione categoria, integrato in home, upload e cronologia; corretti un errore di sintassi in main.dart e un import mancante nel backend; individuata e risolta una discrepanza tra .env e config.py sul nome del database Cosmos

9.4 Ciclo di Vita, Ottimizzazioni e Deploy

Domanda 12: Ho accidentalmente tralasciato i servizi Azure attivi anche se non ho programmato affatto. Forniscimi dei consigli su come posso procedere per continuare a programmare senza dover sprecare i miei token. La procedura è presente nelle slide viste all'esercitazione potresti esplicitarmela e specificare i punti di mio interesse.

Modello usato: Gemini 1.5 Pro

Risposta: In accordo con il documento da te allegato ecco la procedura migliore in aggiunta si può implementare il check di un Test mode che ti permetterà di swicciare agevolmente da applicazione collegato ai servizi azure a uno emulato con risposte di prova prima di portarlo a regime. L'IA ha suggerito l'eliminazione temporanea delle risorse PaaS costose tramite Bicep e l'implementazione architetturale della modalità di test (TEST_MODE=True). Questo ha permesso l'utilizzo di *Azurite* per lo storage, l'emulatore *Cosmos DB* per i metadati, e la generazione di *mock data* locali (es. array

vettoriali riempiti a zero) per isolare lo sviluppo del backend dai costi dell'Intelligenza Artificiale in Cloud.

Domanda 13: L'utente ha caricato il file README_DEPLOY.md, una guida in italiano per il deploy di un backend FastAPI su Azure (containerizzazione con Docker, infrastruttura con Bicep, CI/CD con GitHub Actions, integrazione con Keycloak e frontend Flutter), chiedendo di sistemare il file markdown.

Modello usato: Claude Sonnet 5

Risposta: Il modello ha analizzato il file e prodotto una versione corretta (README_DEPLOY.md), intervenendo su:

- gerarchia dei titoli non uniforme (mix di # e ## per sezioni dello stesso livello);
- refusi ortografici (“deploiare”, “pochè”, “Keycloak”);
- blocchi di codice senza linguaggio specificato per l'evidenziazione della sintassi;
- passaggi elencati come testo semplice invece che come liste puntate/numerate;
- una vulnerabilità nel file Bicep, dove la password amministrativa di Keycloak era scritta come stringa letterale nel codice sorgente nonostante fosse marcata `secureValue` — trasformata in parametro `@secure()`;
- un segnaposto non funzionante (`<tuusername>`) nel workflow GitHub Actions, sostituito con la variabile `${{ secrets.DOCKER_USERNAME }}`.

10. Conclusioni e Sviluppi Futuri

Il progetto ha conseguito con successo tutti gli obiettivi prefissati dalla traccia d'esame, concretizzandosi in un'applicazione cloud-native complessa, scalabile e interamente basata sull'ecosistema Microsoft Azure per la gestione, l'analisi e la ricerca di un archivio di notizie.

L'adozione di un'architettura rigorosamente disaccoppiata (*decoupled*) ha permesso di separare con successo il layer di presentazione — sviluppato in Flutter per garantire un'esperienza utente cross-platform fluida e reattiva — dal layer di orchestrazione e logica di business, gestito tramite un backend asincrono in Python (FastAPI).

Il sistema ha soddisfatto sia i requisiti obbligatori sia quelli facoltativi, integrando un modulo di ingestion multiformato capace di estrarre testo non solo da file strutturati (TXT, Markdown, JSON), ma anche da formati complessi come PDF e DOCX.

L'infrastruttura dati ha dimostrato l'efficacia del paradigma Zero-SQL, delegando la persistenza dei file fisici ad Azure Blob Storage e l'archiviazione dei metadati eterogenei ad Azure Cosmos DB (API NoSQL).

Il cuore innovativo del sistema, rappresentato dal motore RAG (Retrieval-Augmented Generation), ha dimostrato la capacità di orchestrare pipeline di Intelligenza Artificiale avanzate tramite LangChain, Azure AI Search e Azure OpenAI (modelli gpt-4o-mini e text-embedding-ada-002). Il sistema permette interrogazioni semantiche in linguaggio naturale, garantendo sempre la tracciabilità delle fonti e mitigando le allucinazioni tipiche dei Large Language Models grazie a tecniche di Prompt Engineering contestuale e output strutturato (JSON).

Infine, l'infrastruttura di virtualizzazione e la sicurezza delegata a Keycloak (tramite protocollo OAuth2/OIDC) hanno garantito il rispetto delle direttive cloud senza incorrere in violazioni architetturali o superamenti del budget accademico.

10.2 Valutazione delle Scelte Architettureali e Cost Management

La fase di sviluppo ha evidenziato l'importanza critica di un'ingegneria del software consapevole dei costi operativi, un aspetto spesso trascurato in ambito accademico. L'uso di emulatori locali e l'implementazione architetturale della modalità di test (TEST_MODE=True), unita alla gestione dell'infrastruttura come codice tramite script Bicep, ha permesso di condurre collaudi intensivi senza erodere i limiti della sottoscrizione Azure for Students.

Allo stesso modo, la scelta di adottare Keycloak containerizzato tramite Docker ha rappresentato un'elegante soluzione per aggirare l'impossibilità di allocare un tenant Azure Entra ID B2C, rispettando contemporaneamente il principio di sicurezza informatica fondamentale della delega crittografica.

10.3 Sviluppi e Ottimizzazioni Future

Nonostante il sistema sia pienamente funzionale e pronto per la fase di demo, l'architettura a microservizi adottata si presta a molteplici evoluzioni in vista di un ipotetico rilascio in un ambiente di produzione Enterprise.

- **Evoluzione dell'Identity Management (Migrazione PaaS):** in uno scenario libero dai vincoli imposti dall'account universitario, lo sviluppo naturale del sistema prevedrebbe la dismissione del container Keycloak in favore di Microsoft Azure Entra ID B2C. Questa transizione consentirebbe di sfruttare un servizio gestito nativamente dal provider cloud, come anche funzionalità del tipo procedura recupera password, che non è stata implementata per una scelta di tempistica e per privilegiare il perfezionamento di altre funzioni principali nel progetto.
- **Scalabilità Asincrona e Message Queuing (Pattern Web/Worker Role):** attualmente, le operazioni computazionalmente intensive (come l'estrazione del testo dai PDF lunghi e la generazione vettoriale tramite OpenAI) sono gestite tramite coroutine asincrone sul thread principale del backend FastAPI. In un'ottica di altissima scalabilità, il sistema potrebbe essere rifattorizzato applicando il classico pattern distribuito basato sulla separazione tra Web Role e Worker Role: il backend FastAPI (Web Role) si limiterebbe a ricevere l'articolo caricato, posizionando un messaggio in una coda asincrona (es. Azure Storage Queues o Service Bus); un microservizio dedicato (Worker Role), in ascolto sulla coda, si occuperebbe di prelevare il messaggio ed eseguire l'indicizzazione RAG in background, garantendo il bilanciamento del carico (*load balancing*) e prevenendo colli di bottiglia durante l'upload massivo di centinaia di articoli in simultanea.
- **Estensione Multimodale del Motore RAG:** un ulteriore sviluppo dell'applicativo consisterebbe nell'evoluzione della pipeline di ingestion da puramente testuale a multimodale. Sfruttando le capacità di Computer Vision dei modelli Azure OpenAI più recenti, il sistema potrebbe essere istruito per analizzare grafici, tabelle e immagini contenute all'interno dei documenti PDF, generando metadati e vettori anche per i contenuti visivi, arricchendo drasticamente la qualità e la precisione del motore di ricerca semantico.